SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA1409 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. *(BL3)*

Assessment Tool Weightage: 10%

QUESTIONS: **Total Marks:50**

Annexure A

SCENARIO BASED TASK

Code of Conduct:

*I **E.Ragul / 192571032** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

E.Ragul

Annexure A

SCENARIO-BASED TASK

1.Scenario: Parsing Table Conflict in LL(1) While designing an LL(1) parser, a student constructs a parsing table and finds multiple entries in a single cell for a non-terminal and input symbol combination. This leads to confusion during parsing. Analyze how the syntax analyzer uses FIRST and FOLLOW sets in this process, identify the reason for the conflict, and explain how it affects deterministic parsing. Conclude by suggesting what decision should be taken to modify the grammar so that it becomes suitable for LL(1) parsing.

Answer 1:

1.1 Understanding of the Scenario

1. The syntax analyzer is an LL(1) predictive parser: Left-to-right scan, Leftmost derivation, 1 lookahead token.
2. It chooses a production using the parsing table M[A, a] (non-terminal A, lookahead a).
3. In the student's table, one cell holds more than one production.
4. The parser cannot choose a single production, so parsing is not deterministic.

1.2 Use of FIRST and FOLLOW by the Syntax Analyzer

1. FIRST(α) is the set of terminals that can begin a string derived from α (ε is included if α can vanish).
2. FOLLOW(A) is the set of terminals that can appear immediately after A in some derivation ($ for the start symbol).
3. FIRST rule: for each terminal a in FIRST(α), place A → α in M[A, a].
4. FOLLOW rule: if ε is in FIRST(α), place A → α in M[A, b] for every b in FOLLOW(A).
5. Blank cell = syntax error. One entry = deterministic move. Two or more entries = conflict.

1.3 Reason for the Conflict (Key Issues)

1. **Common prefix:** A → αβ | αγ gives a FIRST/FIRST conflict.
2. **Left recursion:** A → Aα | β makes the FIRST sets overlap and can cause infinite looping.
3. **Ambiguity:** an ambiguous grammar can never be LL(1).
4. **Nullable alternative:** if FIRST(α) ∩ FOLLOW(A) ≠ ∅, a FIRST/FOLLOW conflict occurs.
5. The language may need more than one lookahead token.

1.4 Application with Examples Example 1: FIRST/FIRST conflict

```
S → a S | a
FIRST(aS) = {a}     FIRST(a) = {a}
Both productions go into M[S, a]  →  conflict
```

Example 2: FIRST/FOLLOW conflict

```
S → A a
A → a | ε
FIRST(A) = {a, ε}     FOLLOW(A) = {a}
M[A, a] gets A → a (FIRST rule) and A → ε (FOLLOW rule)  →  conflict
```

1.5 Effect on Deterministic Parsing

1. The parser must guess the production and backtrack on a wrong guess.
2. Parsing time may grow from linear to exponential.
3. Errors are detected late and error messages become poor.
4. Semantic actions may be executed more than once.
5. Parser generators report "multiple entries" and the table cannot be used directly.

1.6 Decision: Modify the Grammar

1. **Left factoring:** $A \rightarrow \alpha\beta \mid \alpha\gamma$ becomes $A \rightarrow \alpha A'$ and $A' \rightarrow \beta \mid \gamma$.
2. **Remove left recursion:** $A \rightarrow A\alpha \mid \beta$ becomes $A \rightarrow \beta A'$ and $A' \rightarrow \alpha A' \mid \varepsilon$.
3. **Remove ambiguity** by rewriting the rules.
4. Recompute FIRST and FOLLOW, then rebuild the table.
5. Check the LL(1) conditions: $FIRST(\alpha) \cap FIRST(\beta) = \emptyset$; only one alternative is nullable; if β is nullable then $FIRST(\alpha) \cap FOLLOW(A) = \emptyset$.
6. Reason: the language stays the same, backtracking is avoided and parsing stays linear.
7. If a conflict still remains, switch to an LR(1)/LALR parser.

Modified grammar for Example 1:

```
S  → a S'
S' → S | ε
```

Non-terminal	FIRST	FOLLOW
S	{ a }	{ $ }
S'	{ a, ε }	{ $ }

`FIRST(S) ∩ FOLLOW(S') = {a} ∩ {$} = ∅  →  LL(1) condition satisfied`

1.7 Additional Points

1. LL(1) grammars are unambiguous, not left recursive and left factored.
2. Every LL(1) grammar is also LR(1), but not every LR(1) grammar is LL(1).
3. Advantage of LL(1): the parser is simple, fast and easy to write by recursive descent.
4. Limitation: LL(1) cannot handle left-recursive or ambiguous grammars directly.
5. Tools such as ANTLR and LL(1) table generators report the conflicting cell, which helps in fixing the grammar.

Conclusion

Multiple entries arise from overlapping FIRST/FOLLOW sets. Left factoring and removal of left recursion and ambiguity give one production per cell and deterministic parsing.

OUTPUT

Predictive parsing table:

Non-terminal	a	$
S	S → aS'	
S'	S' → S	S' → ε

```
Input aa$ :  S → aS'  →  match a  →  S' → S  →  S → aS'
             →  match a  →  S' → ε  →  ACCEPTED Every
cell has exactly one entry: the grammar is LL(1).
```

2.Scenario: Incorrect Parse Tree Generation A syntax analyzer generates a parse tree for the expression id + id - id, but the evaluation gives incorrect results due to improper grouping of operators. Analyze how the syntax analyzer constructs the parse tree, identify the issue related to associativity, and explain how grammar rules influence the structure of the tree. Finally, discuss what decision should be taken to enforce correct associativity and ensure accurate parsing.

Answer 2:

2.1 Understanding of the Scenario

1. The input is id + id - id and the evaluation result is wrong.
2. The result depends on how the operators are grouped in the parse tree.
3. Wrong grouping means the grammar allowed a tree with the wrong structure.

2.2 How the Syntax Analyzer Constructs the Parse Tree

1. It reads tokens from the lexical analyzer one by one.
2. Top-down parsers apply productions (derivation); bottom-up parsers apply reductions.
3. Each production creates an internal node, and each token becomes a leaf.
4. The operator at the deeper level is evaluated first.
5. Therefore the grammar rules decide the tree shape and the evaluation order.

2.3 Key Issue: Associativity

1. The grammar E → E + E | E - E | id is ambiguous.
2. It does not say whether + and - group from the left or from the right.
3. The input id + id - id therefore has two parse trees.

```
Tree 1 (correct)   : ((id + id) - id)     left grouping
Tree 2 (wrong)     : (id + (id - id))     right grouping
```

Tree	Grouping	10 - 5 + 3	Verdict
Tree 1	left: (10 - 5) + 3	8	Correct
Tree 2	right: 10 - (5 + 3)	2	Wrong

2.4 Influence of Grammar Rules on the Tree

1. **Left recursion** (E → E + T) makes the tree grow to the left: left associativity.
2. **Right recursion** (E → T + E) makes the tree grow to the right: right associativity (suitable for = and ^).
3. **Separate levels** (E, T, F) give precedence to operators at the lower level.
4. **Ambiguity** gives more than one tree for the same input.
5. In arithmetic, + and - are left associative, so left recursion is required.

2.5 Decision: Enforce Correct Associativity

1. Replace the ambiguous grammar with a left-recursive, unambiguous grammar.
2. For top-down parsing, remove the left recursion and evaluate from left to right.
3. In yacc/bison, declare %left '+' '-' to resolve the conflict.
4. Reason: only one tree is possible, so evaluation is always correct.

```
Bottom-up :  E → E + T | E - T | T
             T → id

Top-down  :  E  → T E'
             E' → + T E' | - T E' | ε
             T  → id
```

2.6 Additional Points

1. Precedence decides which different operator is applied first; associativity decides the order for operators of the same precedence.
2. Left associative: + - * / . Right associative: = and exponentiation (^).
3. A syntax-directed translation or evaluation walks the tree bottom-up, so tree shape fixes the result.
4. Checking a grammar: if any string has two parse trees or two leftmost derivations, the grammar is ambiguous.
5. Ambiguity cannot be detected by a general algorithm, so grammars are designed carefully and tested.

Conclusion

The wrong result is caused by an ambiguous grammar with no associativity rule. A left-recursive grammar forces left grouping, gives a single parse tree and accurate evaluation.

OUTPUT

```
Derivation: E ⇒ E - T ⇒ E + T - T ⇒ T + T - T ⇒ id + id - id
Parse tree:
        E
      / | \
     E  -  T
   / | \   |
  E  +  T  id
  |     |
  T     id   |   id Grouping: ((id +
id) - id)  →  correct
```

3.Scenario: Invalid Expression Handling A syntax analyzer in a compiler receives the token sequence corresponding to the expression a + * b from the lexical analyzer. Although all tokens are valid individually, the parser fails to generate a parse tree and reports an error. In this situation, analyze how the syntax analyzer identifies the error based on grammar rules, determine the key issue in the token arrangement, and explain how parsing techniques such as predictive or shift-reduce parsing handle such cases. Further, discuss what decision should be taken to improve error detection and recovery while maintaining clarity in parsing.

Answer 3:

3.1 Understanding of the Scenario

1. The lexical analyzer gives the tokens: id + * id.
2. Every token is valid on its own, but the order is not valid.
3. The parser cannot build a parse tree and reports a syntax error.
4.

3.2 How the Syntax Analyzer Identifies the Error

1. The parser checks whether the token string can be derived from the start symbol.
2. The grammar requires an operand (T or F) after every +.
3. The token * cannot begin an operand, so no production matches.
4. The error is detected at the first token that cannot extend a valid prefix (viable-prefix property).

```
E  → E + T | T
T  → T * F | F
F  → ( E ) | id
```

3.3 Key Issue in the Token Arrangement

1. Two binary operators (+ and *) are adjacent.
2. The operand between them is missing.
3. Expected tokens after +: id or (. Found: *.

3.4 Predictive (LL(1)) Parsing

LL(1) form of the grammar with its sets:

```
E  → T E'          E' → + T E' | ε
T  → F T'          T' → * F T' | ε
F  → ( E ) | id
```

Non-terminal	FIRST	FOLLOW
E	{ (, id }	{), $ }
E'	{ +, ε }	{), $ }
T	{ (, id }	{ +,), $ }
T'	{ *, ε }	{ +,), $ }
F	{ (, id }	{ *, +,), $ }

Parsing steps (stack top at left):

Stack	Input	Action
E $	id + * id $	E → TE'
T E' $	id + * id $	T → FT'
F T' E' $	id + * id $	F → id
id T' E' $	id + * id $	match id
T' E' $	+ * id $	T' → ε
E' $	+ * id $	E' → + T E'
+ T E' $	+ * id $	match +
T E' $	* id $	M[T, *] is blank → ERROR

3.5 Shift-Reduce (Bottom-Up) Parsing

Stack	Input	Action
$	id + * id $	shift
$ id	+ * id $	reduce F → id
$ F	+ * id $	reduce T → F
$ T	+ * id $	reduce E → T
$ E	+ * id $	shift
$ E +	* id $	ACTION[state, *] is blank → ERROR

3.6 Error Recovery Techniques

1. **Panic mode:** discard tokens until a synchronising token (from FOLLOW or FIRST sets) is found. Here * is discarded and parsing resumes at id.
2. **Phrase-level:** insert the missing operand id before * and continue.
3. **Error productions:** add rules such as E → E + error to the grammar.
4. **Global correction:** find the minimum changes for a correct program (costly, rarely used).

3.7 Decision: Improve Detection and Recovery

1. Keep the grammar unambiguous, so the error position is exact.
2. Use synchronising sets (FOLLOW) with panic mode for predictive parsers.
3. Use phrase-level insertion or error productions for LR parsers.
4. Give clear messages with line and position, for example "operand expected before *".
5. Avoid cascading errors, so that later errors are still found in the same pass.

3.8 Additional Points

1. A syntax error is different from a lexical error (invalid token) and a semantic error (type mismatch).
2. LL and LR parsers detect an error as soon as it occurs; they never shift an invalid token.
3. A good error handler must report the error, recover quickly and not slow down the parsing of valid programs.
4. Yacc/bison use the special token error for recovery rules.
5. Recovery should not create new false errors in the rest of the program.

Conclusion

The error arises from the missing operand between + and *. Both parsing methods find a blank table entry at *. Panic-mode or phrase-level recovery with clear messages keeps parsing clear and reliable.

OUTPUT

```
Syntax error at "*": operand expected after "+"
Recovery 1 (panic mode)  : skip "*"      →  a + b parsed
Recovery 2 (phrase level): insert id     →  a + id * b parsed
```

4. Scenario: Ambiguity in Conditional Statements A syntax analyzer is designed using the grammar S → if E then S | if E then S else S | a. While constructing the parsing table, conflicts arise, and the parser cannot decide which production to apply for certain inputs. In this scenario, explain how the syntax analyzer encounters ambiguity, identify the key issue known as the dangling else problem, and discuss how it impacts parsing. Suggest an appropriate modification or parsing strategy to resolve the ambiguity and ensure correct association of else clauses.

Answer 4:

4.1 Understanding of the Scenario

1. Grammar: S → if E then S | if E then S else S | a.
2. While building the parsing table, one cell gets two productions.
3. The parser cannot decide which production to apply for some inputs.

4.2 How the Syntax Analyzer Encounters Ambiguity

1. Both if-productions start with the same tokens: if E then S.
2. Only after parsing the inner S can the parser see whether else follows.
3. With one lookahead, the choice between the two productions cannot be made.
4. After left factoring, the conflict moves to the token else.

4.3 Key Issue: The Dangling Else Problem

Input: if E1 then if E2 then a else a. The else can join either if:

```
Tree 1: if E1 then ( if E2 then a else a )     else with inner if
Tree 2: if E1 then ( if E2 then a ) else a     else with outer if
```

Two trees for one string means the grammar is ambiguous.

4.4 Parsing Table Conflict

```
S  → if E then S S' | a
S' → else S | ε
```

Non-terminal	FIRST	FOLLOW
S	{ if, a }	{ $, else }
S'	{ else, ε }	{ $, else }

Cell	Entries	Result
M[S', else]	S' → else S and S' → ε	Conflict

In shift-reduce parsing the same problem appears as a shift/reduce conflict on else.

4.5 Impact on Parsing

1. The grammar is not LL(1) and not conflict-free LR(1).
2. The parser may attach else to the wrong if.
3. The program meaning changes, so the program may behave incorrectly.
4. Parser generators report conflicts.

4.6 Decision: Resolution Strategies

1. **Nearest-then rule:** every else matches the nearest unmatched then (used in C, Java).
2. **Table strategy:** keep S' → else S in M[S', else] and remove S' → ε.
3. **Shift-reduce strategy:** resolve the conflict in favour of shift.
4. **Unambiguous grammar:** separate matched and unmatched statements (below).
5. **Language design:** use end if or braces to remove the problem.

```
S → M | U
M → if E then M else M | a
U → if E then S | if E then M else U
```

Reason: M allows else only when every then before it is already matched, so else is always joined with the nearest then.

4.7 Additional Points

1. The dangling else is an example of a shift/reduce conflict in an LR parser.
2. Yacc/bison resolve shift/reduce conflicts in favour of shift by default.
3. Python and languages with end if or braces do not have this problem.
4. Ambiguity does not always mean the language is ambiguous; a different grammar may remove it.
5. The nearest-then rule is a convention, so it must be followed by every compiler of the language.

Conclusion

The dangling else problem arises from an ambiguous grammar. The nearest-then rule, applied through the table or an unambiguous grammar, gives exactly one tree and correct association of else.

OUTPUT

```
Input: if E then if E then a else a $
S → if E then S S'  →  match if, E, then
S → if E then S S'  →  match if, E, then
S → a  →  match a
lookahead else: S' → else S  (inner S', nearest then)
S → a  →  match a
lookahead $: S' → ε  (outer S')
ACCEPTED: else joined with the inner if
```

5.Scenario: Operator Precedence Conflict A parser is designed for arithmetic expressions using the grammar E → E + E | E * E | id. While parsing the input id + id * id, the syntax analyzer produces an incorrect parse tree due to ambiguity in operator precedence. Examine how the syntax analyzer processes this input, identify the issue caused by the grammar, and explain how precedence and associativity rules should be applied. Based on this, suggest what changes must be made to the grammar or parsing technique to ensure correct evaluation.

Answer 5:

5.1 Understanding of the Scenario

1. Grammar: E → E + E | E * E | id. Input: id + id * id.
2. The parser gives a wrong tree, so the evaluation is wrong.
3. The expected grouping is id + (id * id), because * has higher precedence than +.

5.2 How the Syntax Analyzer Processes the Input

Stack	Input	Action
$	id + id * id $	shift
$ id	+ id * id $	reduce E → id
$ E	+ id * id $	shift
$ E +	id * id $	shift
$ E + id	* id $	reduce E → id
$ E + E	* id $	CONFLICT on *: shift or reduce?

1. **Reduce** E → E + E gives (id + id) * id: wrong.
2. **Shift** * gives id + (id * id): correct.
3. The grammar gives no rule to choose, so a shift/reduce conflict occurs.

5.3 Key Issue Caused by the Grammar

1. + and * are at the same level of the grammar.
2. No precedence information is present.
3. No associativity information is present (id + id + id is also ambiguous).
4. The grammar is ambiguous: two trees for one input.

Tree	Grouping	Value (2, 3, 4)	Verdict
Tree 1	id + (id * id)	2 + 12 = 14	Correct
Tree 2	(id + id) * id	5 * 4 = 20	Wrong

5.4 Applying Precedence and Associativity Rules

1. Precedence order (high to low): parentheses, *, +.
2. Associativity: both * and + are left associative.
3. The operator with higher precedence is generated deeper in the tree, so it is evaluated first.
4. Left recursion in a rule gives left associativity.

5.5 Decision: Changes to the Grammar or Parsing Technique

1. **Grammar change:** use one level for each precedence class (E, T, F).
2. **Top-down parser:** also remove left recursion (E → T E', E' → + T E' | ε, T → F T', T' → * F T' | ε).
3. **yacc/bison:** declare %left '+' and then %left '*'. The later declaration has higher precedence.
4. **Operator-precedence parser:** use relations such as + <· * and * ·> +.
5. Reason: only one tree is possible, so the evaluation is always correct.

```
E → E + T | T
T → T * F | F
F → id
```

5.6 Additional Points

1. Precedence climbing and Pratt parsing evaluate expressions using a precedence number for each operator.
2. A conflict is resolved by comparing the precedence of the rule with that of the lookahead token.
3. If precedences are equal, associativity decides: left means reduce, right means shift.
4. Unary minus needs its own higher-precedence rule (%prec UMINUS).
5. Parentheses override precedence because the inner expression is parsed first.

Conclusion

The incorrect tree is caused by an ambiguous grammar without precedence or associativity. Separate grammar levels, or precedence declarations in the parser, give one tree and correct evaluation.

OUTPUT

```
Derivation: E ⇒ E + T ⇒ T + T ⇒ F + T ⇒ id + T ⇒ id + T * F
                        ⇒ id + F * F ⇒ id + id * id
Parse tree:
          E
        / | \
       E  +  T
       |     / | \
       T    T  *  F
       |    |     |
       F    F     id
|    |       id  id
Grouping: id + ( id * id )  →  2 + 3 * 4 = 14 (correct)
```

RUBRICS FOR CALCULATION:

Scenario based assessment

Criteria	Weigh tage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand